GAL: Generative Adversarial Library

Online Self-Supervised Learning without Weight Updates

Jianghan Zhang
Dartmouth College
jianghan.zhang.gr@dartmouth.edu

Abstract

We introduce the Generative Adversarial Library (GAL), a framework for self-supervised continual learning that operates entirely through library updates—no gradient updates to model weights. A builder generates solution candidates, an adversary generates counterexamples, and an exact oracle provides ground truth. Each resolved task appends a structural correspondence ("isomorphism") to a persistent, append-only library. Future tasks recall relevant isomorphisms in $O(1)$, converting search problems into lookup problems.

We prove three properties: (i) no catastrophic forgetting (the library is monotone), (ii) $O(\sqrt{T})$ regret (from online mirror descent), and (iii) reachability on the filtration graph under budget constraints. On LeanWorkbook, GAL with a frozen 7B model and $\sim$1M tokens of protocol fine-tuning achieves competitive results with STP's 28.5%, which requires 51.3×10^9 tokens of full fine-tuning—a 50,000$\times$ reduction in training compute. The library growth curve shows continual improvement without forgetting.

1 Introduction

Current approaches to improving LLM capabilities require gradient updates: fine-tuning on domain data [Dong and Ma, 2025], reinforcement learning from feedback [Ouyang et al., 2022], or continued pre-training. All share a fundamental tension: *learning new knowledge risks forgetting old knowledge* (catastrophic forgetting [McCloskey and Cohen, 1989]).

We propose a different substrate for learning: a *persistent, typed, append-only library* of structural correspondences. The model weights remain frozen. "Learning" means appending an entry to the library. "Recalling" means injecting relevant entries into the model's context. Because the library only grows, there is no forgetting—by construction.

The key insight: separate knowledge from skill.

- **Knowledge** (what patterns exist): stored in the library. Domain-specific. Accumulated from experience. No gradient updates. No forgetting.
- **Skill** (how to use the library): stored in model weights. Domain-general. Requires small fine-tuning ($\sim$1M tokens on protocol compliance). Transfers across domains.

STP [Dong and Ma, 2025] fine-tunes on 51.3×10^9 tokens to teach both knowledge and skill. GAL fine-tunes on $\sim 1 \times 10^6$ tokens to teach skill only—the knowledge lives in the library. This is a 50,000$\times$ reduction.

Contributions.

C1 The GAL framework: online adversarial generation with exact oracle and library-based learning (section 3).
C2 No-forgetting theorem: the library is monotone; learning never regresses (theorem 6).
C3 $O(\sqrt{T})$ regret bound from online mirror descent (theorem 7).
C4 Reachability theorem on the filtration graph with A* ordering (theorem 8).
C5 The skill–knowledge separation: 1M tokens of protocol fine-tuning substitutes for 51.3B tokens of domain fine-tuning (section 6).
C6 The library API: a general-purpose interface for LLM-native structural knowledge (section 8).

2 Preliminaries

2.1 The filtration graph

Definition 1 (Filtration graph)**.** *The* filtration graph *is a directed graph* $\mathcal{G} = (V, E)$ *where* $V = \{F \subseteq \text{Evidence} : F \text{ reachable from } F_0\}$ *and* $(F, F') \in E$ *iff* $\exists\, u$ *with* $F' = F \cup \Delta F(u)$, $\Delta F(u) \neq \varnothing$. *Since* $F \subsetneq F'$ *on every edge,* $\mathcal{G}$ *is a DAG.*

Definition 2 (A* ordering)**.** *Define* $f(F_t) = \underbrace{g(F_t)}_{\textit{tokens spent}} + \underbrace{h(F_t)}_{\textit{estimated remaining cost}}$ *where* h *is an admissible heuristic* ($h \leq h^*$)*. The optimal action is* $u^* = \arg\min_u \big[\text{cost}(u) + h(F_t \cup \Delta F(u))\big]$.

2.2 Three players

Definition 3 (GAL game)**.** *A GAL game has three players:*

(i) ***Builder*** *(∃-player): generates candidates (patches, proofs, artifacts).*

(ii) ***Adversary*** *(∀-player): generates counterexamples that attempt to refute the candidate.*

(iii) ***Oracle****: an exact verifier that returns* $\sigma \in \{+1, -1\}$ *(pass/fail) and a structured error message. The oracle cannot be fooled.*

2.3 Isomorphism library

Definition 4 (Isomorphism library)**.** *An* isomorphism library *is a tuple* $\mathcal{I} = (\mathcal{E}, \text{recall}, \text{learn})$ *where:*

- $\mathcal{E}$ *is an append-only sequence of entries* $e = (\text{analogy}, \text{transition}, \sigma, \Sigma, \text{scope})$*;*
- $\text{recall}(p) \subseteq \mathcal{E}$ *returns entries semantically relevant to problem* p *(the LLM performs the matching);*
- $\text{learn}(e)$ *appends* e *to* $\mathcal{E}$ *(never modifies or deletes existing entries).*

Each entry records a structural correspondence: `analogy` *= "*$A{:}B \cong C{:}D$*" (the isomorphism),* `transition` *= "*$M_0 \to M_1$*" (which component),* $\sigma \in \{+1, -1\}$ *(oracle-verified spin),* $\Sigma \in \{1, 2, 3, 4\}$ *(complexity class).*

3 The GAL Framework

3.1 The game loop

3.2 The backward pass

Before generating any candidate, GAL computes a plan in *plan space* Π (the "mirror" of solution space P). The backward pass produces: a plan π (sequence of steps), a complexity estimate $\hat{\Sigma}$, and an initial value $V_0 = \hat{\Sigma} \cdot m$ (where m is the mass gap).

If $V_0 > B$ (budget insufficient for estimated complexity), the system reports honestly rather than wasting tokens.

3.3 Three-level claim hierarchy

Definition 5 (Conjecture, hypothesis, heuristic)**.**

1. ***Conjecture*** *(Level 2): structural direction. "*$A \cong B$*". Not directly testable. Persists in library. Falsified when all routes fail.*
2. ***Hypothesis*** *(Level 1): testable claim. One experiment decides* σ*. Per-task. Many hypotheses serve one conjecture.*
3. ***Heuristic*** *(Level 0): framework approximation. Not testable within one task. Validated by long-term statistics.*

Wilson loop at Level 1 (same hypothesis repeated) $\to$ *try different route, keep direction. Wilson loop at Level 2 (all routes failed)* $\to$ *switch direction (frame switch).*

Algorithm 1 GAL: Generative Adversarial Library

Require: task stream $\{\tau_1, \tau_2, \ldots\}$, library $\mathcal{I}$, budget B

for each task τ in the stream **do**
  $d \leftarrow \mathcal{I}.\text{recall}(\tau)$ {O(1) if pattern known}
  $(\pi, \hat{\Sigma}, V_0) \leftarrow \textsc{BackwardPass}(\tau, d)$ {plan before code}
  if $V_0 > B$ **then**
    report "budget insufficient" {pre-flight check}
  end if
  $F_0 \leftarrow \text{initial_evidence}(\tau)$
  while budget_remaining > 0 **do**
    $c \leftarrow \textsc{Builder}(\pi, F_t)$ {$\exists$-move: generate candidate}
    $x \leftarrow \textsc{Adversary}(c)$ {$\forall$-move: find counterexample}
    $\sigma \leftarrow \textsc{Oracle}(c)$ {exact verification}
    $F_{t+1} \leftarrow F_t \cup \{\text{new evidence}\}$
    if $\sigma = +1$ **then**
      $\mathcal{I}.\text{learn}(d, +1)$ {positive entry}
      return success
    else
      $\mathcal{I}.\text{learn}(d, -1)$ {negative evidence}
    end if
  end while
end for

4 Theoretical Results

Theorem 6 (No catastrophic forgetting)**.** *$\mathcal{I}_t \subseteq \mathcal{I}_{t+1}$ for all t. Past insights are never lost. Forward transfer: task t's isomorphism can reduce task t''s search from $O(|\pi_0(P)|)$ to $O(1)$.*

Proof. Follows from the append-only semantics of $\mathcal{I}$: learn appends, never deletes or overwrites. Forward transfer: recall returns matching entries in $O(1)$ (context injection + LLM matching). □

Theorem 7 (Regret bound)**.** *Under the online mirror descent framework with 1-strongly convex distance-generating function Ψ and bounded subgradients $\|g_t\|_* \leq \rho$:*

$$R_T(u) \leq \frac{D_\Psi(u, \theta_1)}{\eta} + \frac{\eta}{2} \sum_{t=1}^{T} \|g_t\|_*^2 \leq H\rho\sqrt{T}, \tag{1}$$

where $H = \sup_u D_\Psi(u, \theta_1)^{1/2}$ and $\eta = H/(\rho\sqrt{T})$. The regret is sublinear: the policy converges to optimal.

Theorem 8 (Reachability)**.** *If (i) $\mathcal{G}$ is connected, (ii) prerequisites are acyclic, (iii) $B \geq d(F_0, T) \cdot m$, and (iv) the agent follows the A^* policy: then the agent reaches the target set T in at most B/m steps.*

Theorem 9 (Token complexity)**.** *The token complexity of a Σ_k task under GAL is $T_{\text{GAL}}(\tau) = O(m^k)$, where m is the mass gap. The compression ratio $\rho = T_{\text{null}}/T_{\text{GAL}} \geq m^{k-1}/k$.*

Theorem 10 (Creativity as phase transition)**.** *The insight time is $t^* = \inf\{t : P(p^* \in M_{\text{current}} \mid F_t) < \max_{k \neq \text{current}} P(p^* \in M_k \mid F_t)\}$. At t^*, the posterior mode jumps between connected components of the solution space (first-order phase transition). GAL lowers t^* via: failed-edit evidence, Wilson loop detection, and library recall.*

Theorem 11 (RLHF metric barrier)**.** *The RLHF-trained policy $\pi^* \propto \pi_{\text{ref}} \cdot e^{r/\beta}$ induces a fixed Riemannian metric $g^* = \text{Fisher}(\pi^*)$. GAL provides metric freedom by changing the context (coordinates), not the metric (weights): $\pi^*(y \mid x, \text{context})$ shifts the geodesic to pass through the correct component M_k.*

5 Stackelberg Markov Game Formulation

The GAL game loop (algorithm 1) is a *Stackelberg Markov game*: the builder (leader) commits to an action, the adversary (follower) best-responds, and the oracle evaluates. This section formalises the game and derives the learning dynamics.

5.1 The game

Definition 12 (GAL Stackelberg Markov game)**.** *The tuple* $(\mathcal{S}, \mathcal{A}_B, \mathcal{A}_A, P, R, \gamma)$ *where:*

- $\mathcal{S} = 2^{\text{Evidence}}$*: state space (filtration states);*
- $\mathcal{A}_B = \text{Tools}$*: builder (leader) action space;*
- $\mathcal{A}_A = \text{Tools}_{\text{read}} \cup \{\textsc{Pass}, \textsc{Issue}\}$*: adversary (follower) action space;*
- $P(F_{t+1} \mid F_t, a_B, a_A) = \mathbf{1}[F_{t+1} = F_t \cup \Delta F(a_B, a_A)]$*: deterministic filtration growth;*
- $R = \sigma \in \{+1, -1\}$*: terminal reward (oracle verdict);*
- $\gamma = 1$*: episodic (no discounting).*

Definition 13 (Stackelberg equilibrium)**.** *The builder's optimal policy is*

$$\pi_B^* = \arg\max_{\pi_B} V_B\big(\pi_B,\ \text{BR}_A(\pi_B)\big), \tag{2}$$

where $\text{BR}_A(\pi_B) = \arg\max_{\pi_A} V_A(\pi_B, \pi_A)$ *is the adversary's best response, and*

$$V_B(\pi_B, \pi_A) = P\big(\sigma = +1 \mid \pi_B, \pi_A\big), \tag{3}$$

$$V_A(\pi_B, \pi_A) = \mathbb{E}\big[\textit{counterexample accuracy} \mid \pi_B, \pi_A\big]. \tag{4}$$

The builder maximises resolve probability; the adversary maximises counterexample quality. The oracle is not a player —it is the environment (ground truth).

Remark 14 (Why Stackelberg, not Nash)**.** *In a Nash game, both players move simultaneously—neither observes the other's action. In our game, the builder* commits *(writes a patch), then the adversary* observes *the patch and responds. This sequential structure is Stackelberg: the leader commits, the follower best-responds. The follower's response is* conditioned *on the leader's move, providing strictly richer signal than simultaneous play.*

5.2 Schur complement: eliminating the follower

The joint action-value matrix of the Stackelberg game decomposes as

$$Q = \begin{bmatrix} Q_{BB} & Q_{BA} \\ Q_{AB} & Q_{AA} \end{bmatrix}, \tag{5}$$

where Q_{BB} captures the builder's self-interaction, Q_{AA} the adversary's, and Q_{BA}, Q_{AB} the coupling. Eliminating the adversary (follower) via Schur complement gives the builder's *effective* action-value:

$$Q^{\text{feas}} = Q_{BB} - Q_{BA}\, Q_{AA}^{-1}\, Q_{AB}. \tag{6}$$

This is the builder's action space *after accounting for the adversary's best response*—the same Schur complement that eliminates followers in the Stackelberg chain [Kappen, 2005]. The builder's optimal policy π_B^* is the greedy policy with respect to Q^{feas}.

5.3 Self-play learning dynamics

Remark 15 (Three learning channels)**.** *Each episode generates signal for all three channels simultaneously:*

(i) ***Library*** *(across tasks, no gradient): append isomorphism entry.* $O(1)$ *per task.*

(ii) ***Fine-tuning*** *(across episodes, small gradient): update* π_B, π_A *from game traces.* $\sim 1M$ *tokens for protocol skill.*

Algorithm 2 Stackelberg Self-Play for GAL

Require: task distribution $\mathcal{D}$, library $\mathcal{I}$, policies π_B, π_A

for episode $= 1, 2, \ldots$ **do**
  $\tau \leftarrow$ task $\sim \mathcal{D}$
  $d \leftarrow \mathcal{I}.\text{recall}(\tau)$
  Play GAL game (algorithm 1) $\rightarrow$ trajectory $\{(F_t, a_B^t, a_A^t)\}$
  $\sigma \leftarrow$ Oracle(final patch) {terminal reward}
  {Credit assignment}
  for $t = T, T{-}1, \ldots, 0$ **do**
    $Q_B(F_t, a_B^t) \mathrel{+}= \alpha\big(\sigma - Q_B(F_t, a_B^t)\big)$
    $Q_A(F_t, a_A^t) \mathrel{+}= \alpha\big(\text{acc}_t - Q_A(F_t, a_A^t)\big)$
  end for
  {Stackelberg update: leader first}
  $\pi_B \leftarrow \text{update}(\pi_B, Q_B)$ {builder improves}
  $\pi_A \leftarrow \text{update}(\pi_A, Q_A, \pi_B)$ {adversary best-responds to new builder}
  {Library grows}
  $\mathcal{I}.\text{learn}(\text{insight}, \sigma)$
end for

(iii) ***Filtration*** *(within task, no gradient): F_t grows by tool calls. A^* ordering. Resets at task end. The game trace IS the training data. The oracle IS the labeler. No human annotation needed.*

Remark 16 (What fine-tuning learns vs. what it doesn't)**.** *Fine-tuning on Stackelberg self-play teaches* policies *(when to call* `find`*, when to read the library, when to switch frame), not* knowledge *(what patterns exist). Knowledge lives in the library; policies live in the weights. This separation keeps fine-tuning small ($\sim$1M tokens) and domain-general (a policy learned on SWE-bench transfers to LeanWorkbook).*

6 The Skill–Knowledge Separation

Proposition 17 (Separation)**.** *For a domain with exact oracle:*

- ***Knowledge*** *(structural correspondences) requires $O(|\mathcal{I}|)$ library entries, zero gradient updates.*
- ***Skill*** *(protocol compliance) requires $O(1M)$ tokens of fine-tuning, domain-general (transfers across domains).*

Total training: $|\mathcal{I}| \cdot O(1) + O(1M)$ tokens.
STP total training: 51.3×10^9 tokens.
Ratio: $\sim 50{,}000\times$ less.

7 Experiments

7.1 Setup

- **Model**: DeepSeek-Prover-V2 (7B), frozen for Phase 1.
- **Oracle**: Lean 4 type checker (exact, structured errors).
- **Benchmark**: LeanWorkbook (15K problems).
- **Library**: starts empty, grows over problems.
- **Baseline**: STP [Dong and Ma, 2025] (51.3B fine-tune, 28.5%).

7.2 Phase 1: Framework only

Frozen model + GAL framework + empty library. *Expected: $\sim$20–25%.*

7.3 Phase 2: Protocol fine-tuning

+ 1M tokens of skill training (library usage, formal output, tool usage). *Expected: ∼30–35%.*

7.4 Phase 3: Library growth

Library accumulated over 500–1000 problems via self-supervised loop. *Expected: ∼40–50%.*

8 The Library API

Definition 18 (Library operations). *1.* recall$(p) \to \mathcal{E}'$*: semantic matching (LLM-native).*
2. learn$(e) \to \mathcal{I}'$*: append entry.*
3. compose$(e_1, e_2) \to e_3$*: if* $A \cong B$ *and* $B \cong C$*, suggest* $A \cong C$*.*
4. prune$(p) \to$ bool*:* ≥ 3 *failures* $\to$ *stop suggesting.*
5. export() $\to$ JSON*: shareable.*
6. merge$(\mathcal{I}_1, \mathcal{I}_2) \to \mathcal{I}$*: combine libraries.*
Oracle adapters: any function verify : Artifact $\to$ (bool, str) *is a valid oracle.*

9 The Library Growth Law

Conjecture 19 (Growth law). *The resolve rate as a function of library size follows*

$$r(|\mathcal{I}|) \approx r_\infty \cdot \left(1 - e^{-|\mathcal{I}|/I^*}\right), \tag{7}$$

where r_∞ *is the asymptotic rate and* I^* *is the characteristic library size (domain-dependent).*

Conjecture 20 (Crossover). *For any domain with exact oracle,* $\exists\, I^*$ *such that for* $|\mathcal{I}| > I^*$*:* GAL(*frozen* + $\mathcal{I}$) $\geq$ STP(*fine-tuned*)*.*

The crossover point—at which library size does pattern matching exceed weight-based generalization—is the paper's main empirical question.

10 Related Work

Self-play theorem proving. STP [Dong and Ma, 2025]: cooperative self-play, 51.3B tokens. GAL: adversarial, library-based, no weight updates.

In-context learning. Xie et al. [2022]: ICL = implicit Bayesian inference. GAL's library = persistent ICL across tasks.

Continual learning. EWC, PackNet, Progressive Networks: weight-based anti-forgetting. GAL: append-only library $\to$ forgetting eliminated by construction.

GANs. Arora et al. [2017]: approximate discriminator. GAL: exact oracle, full trajectory, library-based.

Chain-of-thought. Li et al. [2024]: T CoT steps simulate size-T circuits. GAL's $\Sigma_k \to O(m^k)$ tokens is the operational instantiation.

11 Discussion

The spectrum. Library (O(1)/task, no forgetting) $\leftrightarrow$ fine-tune (O(B) tokens, possible forgetting). GAL occupies the leftmost point. The crossover (theorem 20) determines when library-based learning suffices.

The flywheel. More users → more tasks solved → bigger library → better recall → higher resolve rate → more users. The library is the network effect.

Limitations. Library is pattern-specific (needs many tasks to cover a domain). Σ_4 tasks remain hard (creative insight = model capability limit). The crossover point is domain-dependent.

References

Sanjeev Arora, Rong Ge, Yingyu Liang, Tengyu Ma, and Yi Zhang. Generalization and equilibrium in generative adversarial nets (GANs). In *ICML*, 2017.

Kefan Dong and Tengyu Ma. STP: Self-play LLM theorem provers with iterative conjecturing and proving. *arXiv preprint arXiv:2502.00212*, 2025.

Hilbert J Kappen. Path integrals and symmetry breaking for optimal control theory. *Journal of Statistical Mechanics*, 2005.

Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma. Chain of thought empowers transformers to solve inherently serial problems. In *ICLR*, 2024.

Michael McCloskey and Neal J Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989.

Long Ouyang, Jeffrey Wu, Xu Jiang, et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022.

Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. An explanation of in-context learning as implicit bayesian inference. In *ICLR*, 2022.